OREBiT Chat + Rain Issue Audit

Consolidated review of the 102 findings in OREBIT-CHAT-RAIN-FIXES.html, checked against the user/admin backend source snapshots.

Total findings	Resolved	Pending validation
102	**100**	**2**

Pending findings

N-098: muted-delivery p95 regression needs a quiet-host comparison between muted and control runs. **N-102:** 2,500-5,000 online load variability needs repeated Linux runs with GC and connection-pool metrics.

These are evidence gates, not claimed fixes. No quiet Linux load run completed in this snapshot: Docker Desktop Linux engine was unavailable, no local PostgreSQL/Redis services were found, and the backend did not start. The user and admin frontend development servers returned HTTP 200 at localhost:3000 and localhost:5173.

Repository and validation evidence

Area	Result
Branch / commit	index / fe95d59; pushed to origin/index
User backend	Full chat suite recorded: 851 passed, 1 skipped; after archived-report change, targeted e2e + DB specs: 28 passed. Typecheck passed.
Admin backend	Full chat suite recorded: 441 passed; after archived-report change, DB specs: 13 passed. Focused OpenAPI specs: 177 passed. Full typecheck has unrelated pre-existing errors.
Frontend builds	User and admin production builds passed.
Runtime	User and admin frontends HTTP 200; backend health endpoint unavailable because Docker engine was stopped.
Migrations	All 69 migrations applied to scratch DB orebit_chat_audit; SQL verification reported 22 indexes.
Batch validation	The issue list below is grouped in consecutive five-issue batches. Test evidence is suite-level plus focused regressions, not one isolated test per finding.

Scope: HTML "unproven" flags are source annotations, not live completion status. The two performance findings remain open until required measurements are recorded.

Batch 01: N-001 - N-005

Five-issue review group

N-001 | RESOLVED P0
Rain SQL uses columns that do not exist, so rain cannot read its settings or schedule

Finding: The user backend asks for rain.config.config and for rain.schedule columns currency, pot_minor, pot_usd_minor, locked_by and locked_at. The migrations created document, next_currency, next_pot_minor, next_pot_usd_minor, lock_owner and lock_until, and no migration renames them. Every one of these queries fails on a migrated database, while the admin backend still uses the real names.

Implemented fix: Put the real column names back in these queries (the v2 SQL at 254f350 had them) and add one rain test on a real Postgres.

N-002 | RESOLVED P0
Saving a rain always fails because recipient_count is required and never sent

Finding: The rain.runs table requires recipient_count, but insertRun does not list that column, so the database refuses the row. The insert shares a transaction with the wallet move, so the money rolls back and no rain can ever finish. This stays broken even after the column names above are fixed.

Implemented fix: Add recipient_count to insertRun with the number of shares.

N-003 | RESOLVED P0
Rain recipients are paid but never recorded

Finding: All three payout paths move the wallet and call insertRun, but none calls insertShares. Its only caller is settleVerified, and nothing calls settleVerified, so rain.recipients stays empty. With no share rows the alert says 0 people, receivers get no event or reward, history is empty and a revert can never find the money.

Implemented fix: Call insertShares(tx, run.id, run.shares) in all three transactions and delete settleVerified.

N-004 | RESOLVED P0
Reverting a whole rain does nothing

Finding: A whole-rain revert passes null for the user list, and the code turns null into an empty list. The query then filters user_id = ANY of nothing, finds no rows and stops. A single revert ends FAILED, and a bulk revert ends DONE with every rain skipped, so the admin sees success while no money moved; only the RECIPIENT filter works. Decision update: on each user rain revert the admin picks refund the rainer, or do nothing, where no balance moves. The path where recipients lose money and the platform keeps it must go.

Implemented fix: Keep null as "all open shares" in takeShares, and add a Postgres test for single and bulk reverts.

N-005 | RESOLVED P1
A revert can re-take shares already reverted, and leaves empty shares open forever

Finding: takeShares is now a plain SELECT with no reversed_at IS NULL filter, and it never uses previouslyReversed. So a whole revert after a partial one tries the same debit key again, and the unique ledger key rolls the whole revert back. A recipient holding 0 under "till 0" gets no debit, so the share stays open, the rain stays PARTIALLY_REVERSED for ever, and reversed_job_id is no longer written.

Implemented fix: Bring back the v2 UPDATE ... WHERE reversed_at IS NULL RETURNING shape, and store reclaimed 0 for shares with nothing to take.

Batch 02: N-006 - N-010

Five-issue review group

N-006 | RESOLVED P1
Rain lists return raw BigInt and crash

Finding: The service returns database rows directly, and those rows hold BigInt values such as potMinor and amountMinor. JSON cannot write BigInt and the app has no serializer for it, so the route answers a server error once a row exists. The v2 mapping to cut decimal text was dropped.

Implemented fix: Bring back the view mapping so these fields leave as cut decimal strings.

N-007 | RESOLVED P1
A rain status change can wipe admin settings or lose a save

Finding: patchStatus reads the config, which silently becomes the built-in defaults when the row is missing or does not parse, then writes the whole document back. It has no lock and no transaction; v2 used FOR UPDATE and a strict parse. Once the column names are fixed, one bad field plus one staff switch replaces every admin rain setting, and two saves at once lose one.

Implemented fix: Refuse when the row or parse fails, and restore the transaction with FOR UPDATE that changes only status.

N-008 | RESOLVED P1
Ad refund treats every post and pinned minute as equal value

Finding: The refund is price times unused units over all units, where one post and one pinned minute count the same whatever each cost. The stored breakdown lines are ignored. The same rule sits in both backends, so an advertiser can get far more than the true unused value back, limited only by the fee. Decision update: pins are no longer refunded and posted ads are final, so only posts not yet sent are valued, at the price paid per post.

Implemented fix: Value each part from its own price: posts share the post total, pins share the pin total. Use the same rule in U and A.

N-009 | RESOLVED P1
A short database error can kill the whole chat server (unhandled promise rejections)

Finding: The 15 second money tick calls settings.global outside any try/catch, and kick only has try/finally, so a failed settings read rejects a promise nobody catches. The admin-signal listener also calls broadcastMessages with .then and no .catch (two places). No unhandledRejection handler exists anywhere in src, and current Node ends the process on one.

Implemented fix: Wrap pass in try/catch inside kick and log; add .catch with a log to both broadcastMessages calls.

N-010 | RESOLVED P1
Delivering one group message costs work in proportion to the number of members

Finding: Per non-system message the server reads the whole member list, runs one preferences query with an IN list of all member ids, and calls the time-zone port once per member (the port has no batch form, so the Auth owner will build N lookups). It then runs quiet-hour checks per member (a new Intl formatter each time) and awaits all of it before the sender gets an ack. Groups over 20,000 members are not cached and the member query is `LIMIT 100000`, so members past 100,000 silently get nothing although a channel may hold 1,000,000.

Implemented fix: Change the port to a batch `timeZones(ids)` now; read preferences and zones only for members with an active quiet window and cache them per member for minutes; cache one Intl formatter per zone; deliver after the ack; replace the silent `LIMIT` with an explicit "too large" path. Add a test that counts statements and port calls for 10 and 5,000 members.

Batch 03: N-011 - N-014

Five-issue review group

N-011 | RESOLVED P1
The payment PIN is skipped when no verification provider is bound

Finding: `hasPaymentPin` returns false when the Verification port is not bound (or lacks the optional method), and gifts and ad purchases treat that as "this user has no PIN". Nothing in the repo binds the port. Once the Transfer and Charge ports are bound and the PIN port is forgotten, a user who turned the PIN on still pays with no code.

Implemented fix: Make `hasPaymentPin` required on the port and refuse gifts and ads ("feature unavailable") when the verification port is unbound outside the stand-ins.

N-012 | RESOLVED P2
Same ad rooms get a different price by the order sent

Finding: The free room is "index 0 if ordinary", not "the first ordinary room". So the same rooms sent in another order cost a different price, and percent adds change too.

Implemented fix: Price ordinary rooms by count (skip one), in a fixed order.

N-014 | RESOLVED P2
An ad charge can succeed on an ad already marked FAILED

Finding: If a charge hangs past the payment wait, the sweeper can get NOT_PAID and mark the ad FAILED, and then the charge lands. activate returns false, the code returns the FAILED ad with no alert, and the refund query needs CANCELLED, so nothing pays it back.

Implemented fix: When a paid charge meets an ad that is not PENDING, log an error and refund by reference.

N-097 | RESOLVED P1
Muting different popular authors collapses the room

Finding: The muted-author fix works only when every muter is the same set of people. Each different set costs one extra frame per flush, so normal muting brings back the old failure.

Implemented fix: Never split the broadcast by mute set. Send the batch once and filter for the few who muted.

N-013 | RESOLVED P2
A paid ad post counts as used even when no room received it

Finding: Rooms that stopped taking ads are skipped and send errors are only logged, then recordPosted always runs. The ad can reach COMPLETED with nothing shown and no refund. The owner may prefer a per-room share.

Implemented fix: Count the post only when at least one room got it, or refund the missed part.

Batch 04: N-015 - N-019

Five-issue review group

N-015 | RESOLVED P2
Zero-reward ads block the ad reward queue

Finding: When the reward is 0, award returns before setRewards, so rewards stays {} and the row matches for ever. The query has LIMIT 50 and no ORDER BY. After 50 such ads, later ads may never get EXP.

Implemented fix: Store a zero-reward marker on the row, and add ORDER BY.

N-016 | RESOLVED P2
Ad EXP can be lost for good, and the cancellation fee earns EXP

Finding: A failed award stores awarded:false, so rewards is no longer {} and it is never retried. The cancel path awards once and swallows a thrown error. usedUsd is price minus refund, so the fee counts as used spend.

Implemented fix: Retry rows with awarded=false, and work out the used amount before the fee.

N-017 | RESOLVED P2
Two different CHAT_PREMIUM_PORT symbols exist

Finding: The name is exported from two files with separate Symbol values and two different PremiumPort interfaces. Binding one, as chat.module.ts says, leaves the other unbound, so hasDisableAllAds or activePlanIds stays unbound.

Implemented fix: Keep one symbol and one interface.

N-018 | RESOLVED P2
The repeat USD cap applies per repeat, not to the whole ad

Finding: The owner example says the total discount can never pass the chat maximum, but the code applies the USD cap to each repeat. Whether the cap means per ad is an open owner question.

Implemented fix: Apply the cap to the total discount, once the owner confirms.

N-019 | RESOLVED P2
Admin ad cancel and revert send no AD_UNPINNED

Finding: An ad change only kicks the money loop, and syncPins announces only expired and started slots. Open clients keep showing a cancelled pin until reload, while the U group revert does announce it. Whether this is a regression from v2 was not checked.

Implemented fix: On an ad change, unpin rooms whose placement was cancelled, or have syncPins report it.

Batch 05: N-020 - N-024

Five-issue review group

N-020 | RESOLVED P2
New ad features have no tests

Finding: A search of tests finds no check of the blacklist, banned words, delisted coin, hasDisableAllAds, recordAdSpend, discounts or cancellationFee, though the commit claims them. tests/social/** is also left out of npm test, so only test:chat runs them.

Implemented fix: Add the verify tests named in each of these FR items.

N-021 | RESOLVED P2
A failed or crashed ad revert loses the EXP take-back

Finding: markAdRewardsRevert runs only after the whole page loop. Ads already cancelled leave adCandidates, so neither a takeover nor a new admin run can reach them. The refund is paid and the EXP is kept.

Implemented fix: Set revert_rewards inside the cancel transaction, and handle COMPLETED ads separately.

N-022 | RESOLVED P2
The admin gets a revert job id and can never read the job

Finding: A group revert returns a job id, but no admin route reads the status, result or error of money_revert_jobs. A FAILED job is therefore invisible.

Implemented fix: Add a list and a detail route on the job table.

N-023 | RESOLVED P2
A LEVEL group with no value reverts everyone who has a level

Finding: value defaults to empty, and value || 0 makes the rule level >= 0, which every levelled user passes. group also counts as a named filter, so the "revert ALL" guard passes.

Implemented fix: Require a value for LEVEL in the admin schema, and remove || 0.

N-024 | RESOLVED P2
Gift values with too many decimals are rounded when saved

Finding: The gift insert passes values as ::numeric into DECIMAL(18,8) and DECIMAL(36,18) columns, and Postgres rounds on assignment. The rule is cut, never round.

Implemented fix: Pass usdStored and cryptoStored before the insert.

Batch 06: N-025 - N-029

Five-issue review group

N-025 | RESOLVED P2
Gift money work can sit idle with no refusal and no log

Finding: Gifts start with processingStaleMinutes null, stuck gifts are then skipped silently, and missingMoneyNumbers leaves that setting out. Gift reverts also depend on four ads.* numbers, so while one is unset the admin revert answers success and nothing runs.

Implemented fix: Refuse gift start and admin reverts while these numbers are unset, and add the setting to the list.

N-026 | RESOLVED P2
A late completion leaves a FAILED gift while confirm says COMPLETED

Finding: If the transfer call runs past the stale limit and status says NOT_FOUND, the sweeper marks FAILED, and then the transfer completes. completeTransfer returns false, the result is ignored and the sender gets COMPLETED. The money moved, no notice goes out and admin revert cannot reach the row.

Implemented fix: Use the boolean result, and make NOT_FOUND void the reference at the provider.

N-027 | RESOLVED P2
Internal user ids go back to the rainer, and a won rain is not tagged

Finding: The answer to /rain lists recipient UUIDs and the tier, which also shows who is Premium. No tag row is written for winners, so a won rain does not reach the tagged feed. Both are latent until recipients are recorded.

Implemented fix: Return usernames or a count, and add feed rows for winners.

N-028 | RESOLVED P2
Manual "rain as OREBiT" takes no admin choices

Finding: V2-16 says the admin chooses currency, USD amount, room and recipient count each time. The route has no body and the job makes a random pick from settings.

Implemented fix: Add these fields to the job, validate them and use them.

N-029 | RESOLVED P2
Reason is dropped on ad reverts and not kept on gifts

Finding: Ad reverts use only dto.note, so with just a reason the note becomes the hidden text "Reverted by admin", the audit note is null and the group job loses it. The gift path accepts reason and never writes it. Two fields do one job.

Implemented fix: Use one required reason everywhere, and drop the literal.

Batch 07: N-030 - N-034

Five-issue review group

N-030 | RESOLVED P2
The admin repeat filter cannot match finished series

Finding: The admin filter offers FINISHED while the database and user code use DONE. FINISHED returns nothing and DONE is refused.

Implemented fix: Use DONE.

N-031 | RESOLVED P2
Reports on archived DMs lose the message

Finding: Reports join only the live chat.dm_messages table and no code reads chat_archive. After a DM month moves, staff see no content and an old message cannot be reported. The exact archive-read scope is an open owner question.

Implemented fix: Read chat_archive in the report joins, once the owner sets the scope.

N-032 | RESOLVED P2
Rain cancel lost its owner check and audit row

Finding: The old code checked initiatorId, refused other statuses and wrote a CANCELLED audit row. Now user is ignored, expireOne runs on the id, and the answer is always { cancelled: true }. Reach is limited today because rain no longer creates PENDING_VERIFICATION rows.

Implemented fix: Delete the route with the verification layer; if kept, restore the owner check and the audit row.

N-033 | RESOLVED P2
Member list paging skips users whose names have digits after an underscore

Finding: The sort uses localeCompare but the cursor walk uses the plain > operator, and the two orders differ. Names a_b, a1, a2, ab, a_c, a0 with limit 1 return a_b, a_c, ab; a0, a1, a2 are never shown.

Implemented fix: Use one comparison for sort and cursor: plain < and > on the lower-cased name.

N-034 | RESOLVED P2
Socket room moves are not serialised: ghost presence and a missed second tab

Finding: The send path has a per-socket chain but join_room does not. Two close join_room events do not leave the first room, so presence keeps a ghost count and the socket stays in both channels. A reevaluate running across a disconnect calls rooms.leave after handleDisconnect already left, and socket.data.roomId is not cleared, so a second tab is uncounted and drops off the rain recipient list.

Implemented fix: Chain room moves per socket like sends; clear socket.data.roomId in handleDisconnect.

Batch 08: N-035 - N-039

Five-issue review group

N-035 | RESOLVED P2
A database error at handshake tells every client to sign in again

Finding: Any error that is not a HandshakeRefusal (sessionLive, findChatUser or preferences read failing) becomes 'unauthenticated', while session_store_unavailable exists to tell a client to retry.

Implemented fix: Answer session_store_unavailable for unexpected errors.

N-036 | RESOLVED P2
Any restriction change empties the whole facts cache

Finding: access.service clears all cached user facts when a 'restrictions' change arrives. Every staff warn, ban or restriction therefore makes every online user refetch 2 queries and 5 port calls on the next room list or access check. Only staff entries can be stale.

Implemented fix: On 'restrictions' delete only entries whose facts.staffRole is not null.

N-037 | RESOLVED P2
A scheduled one-switch room change resets the other content controls to defaults

Finding: At due time the user backend re-parses the stored patch with a zod schema whose nested fields all have defaults, so one key becomes nine keys; the jsonb || then replaces the whole contentControls object. Global's commands and ads switches are turned off.

Implemented fix: Parse scheduled patches with partial nested objects without defaults, or apply nested jsonb_set per key.

N-038 | RESOLVED P2
One bad scheduled change blocks every later one

Finding: Only schema failures are caught and cancelled. A database error (slug taken since scheduling, deleted region FK) aborts the transaction, the row stays SCHEDULED, and the same first row fails every tick, so no later change applies.

Implemented fix: Catch per change (savepoint or own transaction) and cancel it with a reason.

N-039 | RESOLVED P2
An approved permanent restriction can never run

Finding: When a higher role approves a REPORT-mode staff request for a permanent restriction, carryOut drops the permanent flag and passes no length, so restrict answers validation_failed and the request reopens.

Implemented fix: Pass permanent: true in carryOut when payload.permanent === true.

Batch 09: N-040 - N-044

Five-issue review group

N-040 | RESOLVED P2
A group ban also hits staff of equal or higher rank

Finding: restrictionTargets applies the group rule to every member including the chat ADMIN and the issuer; the enforcement in content.service has no rank check. Staff may act only on users ranked lower.

Implemented fix: Skip group rows for users whose rank is at or above the issuer's stored role (createdByRole).

N-041 | RESOLVED P2
A group ban only blocks room text, not what a ban blocks

Finding: The MUTE rule is checked only in content.service. assertNotBanned (ads, gifts, DM cards, DM groups, upload, reports, retract, commands) reads chat.user_bans only. The refusal reason is chat_muted, a word V2-32 removed for staff.

Implemented fix: Add one group-rule check in assertNotBanned (needs user facts) or a named restriction kind.

N-042 | RESOLVED P2
Staff penalty list hides the newest restrictions and can skip bans

Finding: The list slices the 500 oldest rows (rows arrive oldest first) with no next page, and the ban cursor has millisecond precision and no id, so bans at a page edge are skipped or repeated.

Implemented fix: SQL keyset page on both lists with a {createdAt,id} cursor and TIME_TEXT.

N-043 | RESOLVED P2
Reporting an admin-made ad fails with a server error

Finding: For an admin ad the creator is null, so reported_user_id becomes the all-zero id. No identity.users row for it was found, the foreign key fails and isUniqueViolation is false, so the answer is 500.

Implemented fix: Allow a null reported user for ads, or store the ad id.

N-044 | RESOLVED P2
Staff routes and group ownership transfer accept only the internal UUID, not a username or public code

Finding: Staff warn, ban, penalties and room-access routes and `POST dm/groups/:id/transfer` validate `userId` as a UUID. Other group routes accept a username. So a client that only knows usernames cannot transfer ownership, and staff cannot warn or ban by username or login code (admin lookups can).

Implemented fix: One resolver (username, public code, UUID) for staff routes; transfer by username like the other group routes (the service already resolves a handle).

Batch 10: N-045 - N-049

Five-issue review group

N-045 | RESOLVED P2
The new DM member cache is bounded by entries not rows, can store stale data after a write, and is per process

Finding: The cache allows 5,000 entries of up to 20,000 rows each (100 million row objects possible) and `members` copies the array on every call. A `members` query that started before a ban or leave can store its old result after the invalidate and serve it for 5 s, so a removed member can still receive messages. A second server would be up to 5 s behind.

Implemented fix: Bound by total rows with LRU eviction and return the readonly array without copying; use a version counter per conversation (store only if unchanged); add Redis pub/sub invalidation before a second server (D-34).

N-046 | RESOLVED P2
A user with a DM ban is refused inside chats that already exist

Finding: Every `send` calls `assertMayDmStranger`, not only a new chat. In an existing chat where the other person does not follow the sender, each message is refused with `RESTRICT_DM`. V2-48 says a DM ban stops starting new DMs with non-followers only; this also breaks V2-42 (a restricted decliner cannot write first).

Implemented fix: Apply the restriction only when a conversation is created (or on the first message of a request), not on every send.

N-047 | RESOLVED P2
The unread badge runs one correlated EXISTS per conversation over a monthly-partitioned table

Finding: FR-126 suggested a stored `last_other_message_at`; the fix instead runs one EXISTS probe per conversation, each touching every live monthly partition of `dm_messages` (about 13). The badge is polled, so 2,000 conversations means tens of thousands of index probes per call. Not measured.

Implemented fix: Keep an `unread` counter or `last_other_message_at` on `dm_members` updated at send, or pre-filter on `last_message_at`.

N-048 | RESOLVED P2
A lower role can overwrite a ban's rank and then lift it

Finding: After a ban the member row is BANNED with role MEMBER, so the outrank check in the ban path passes for any staff role. `banMember` then upserts the ban row and overwrites `banned_by_role`, so a lower admin takes over the ban and can unban it. The FR-203 unban rank rule can be beaten with two calls.

Implemented fix: In the ON CONFLICT only update when the new rank is at least the stored rank, or refuse with "already banned"; add the test.

N-049 | RESOLVED P2
Creating a group with invitees can fail after the group was already created

Finding: `POST chat/dm/groups` creates the group, owner row and "created" line, then invites; if an invite limit is unset (fresh install) or an invite refusal throws, the caller gets 403 but the group stays. The user retries and burns the create limit with duplicate groups.

Implemented fix: Check the invite settings before creating, or return the created group with the invite refusal as a field.

Batch 11: N-050 - N-054

Five-issue review group

N-050 | RESOLVED P2
Every group invite call loads the whole member list

Finding: The invite service calls `members(groupId)` (up to 100,000 rows with user joins; cache keeps only 20,000). A 50,000-member channel reads 50,000 rows per invite; beyond 100,000 the ACTIVE check misses people and they are invited again.

Implemented fix: Query only the invitees (`user_id = ANY(...)`), like `bannedAmong` already does.

N-051 | RESOLVED P2
An unknown or suspended username falls back to the raw text and the database cast fails with 500

Finding: Group remove, timeout, role, ban and transfer fall back to the typed text when the handle does not resolve, then `${userId}::uuid` throws; `resolveUserHandle` only finds ACTIVE users, so suspended members cannot be removed by name; a ban with a valid unknown UUID writes a row for a nonexistent user (no FK).

Implemented fix: If the handle does not resolve answer "not a member" (404); resolve members through the member row, not the ACTIVE-user lookup.

N-052 | RESOLVED P2
Inviting the same person again re-sends the live pop-up

Finding: A repeat invite returns the existing id before the window count, so the per-window limit is never charged, yet every call still pushes a live event with the caller's group name to each invitee.

Implemented fix: Push the event only when a new invite row was created.

N-053 | RESOLVED P2
Moderation of a group name is skipped when a private group becomes public

Finding: The check runs only when `name` or `description` is in the request, so flipping to PUBLIC leaves a banned word in the stored name unchecked (V2-51). It also runs before the permission checks, so a member with no rename right takes an offence.

Implemented fix: When the result is public moderate the stored name and description; run permission checks first.

N-054 | RESOLVED P2
Admin sees only the oldest 50 messages of a ticket and cannot page

Finding: The FR-132 fix added a hard `LIMIT 50` oldest-first with no cursor, and the service passes no limit and returns no cursor. Message 51 onward (the latest user replies) can never be read by support. The old code was unbounded, so this is worse than before for long tickets.

Implemented fix: Return `{items,nextCursor}` with the `(created_at,id)` keyset (as the user side does), default 50, max 100; accept `cursor` and `limit` in `TicketQuerySchema`.

Batch 12: N-055 - N-059

Five-issue review group

N-055 | RESOLVED P2
Reconnect catch-up anchor loses microseconds, so the "strictly after" key is not strict

Finding: The anchor row's `created_at` is read as a JS Date (milliseconds) and compared with microsecond `timestamptz`, so the anchor itself and rows in the same millisecond are returned again. With `limit=1` the page is the anchor again and `nextCursor` never advances.

Implemented fix: Select the anchor with `TIME_TEXT('created_at')` (already imported) and compare against the text, as `history` does for cursors.

N-056 | RESOLVED P2
Support stand-in enforces its own hard-coded 3-open-tickets cap, has an empty write guard and no port

Finding: Chat invented `MAX_OPEN_TICKETS_PER_USER: 3` (not an admin setting) and refuses with `limit_reached`; it is check-then-insert with no lock. The owner said ticket limits belong to the Support Team (FR-025, FR-132, V2-52) and V2-146 says no built-in numbers. `assertMayWrite` is an empty method with a comment about locks that do not exist, and `SupportTicketPort` does not exist. The subject length is also checked twice with contradictory limits (200 in the service, 120 in DTO and DB).

Implemented fix: Delete the cap (or hold it as an admin setting that refuses when unset, after asking the owner); add the port interface; delete the empty `assertMayWrite` and unused injected deps; use `SUPPORT_SUBJECT_MAX`.

N-057 | RESOLVED P2
The "five parallel taps" ad test fails every time on a real database

Finding: Five of five runs fail at `expected +0 to be 1`: the ad is paid and ACTIVE with `posted_count` 0 and no AD message when the call returns. Likely a lost first-post race (the winner claims the row with SKIP LOCKED while another call writes it, so nobody posts until the money loop), or a test that expects an immediate post. The developers' handoff says complete. Money is not lost if the loop runs.

Implemented fix: Decide the intended behaviour first, then retry the claim once after `activate` or run `moneyPass` in the test before asserting; run 10 times in a row.

N-058 | RESOLVED P2
The internal ban reason can reach the banned user when a notice is replayed

Finding: The admin BAN action row stores `note: dto.internalReason`; if the chat server missed the live notice and replays it after the LISTEN connection returns, the notice carries that note (also for restriction and warn replays). V2-104 says only the user reason is shown.

Implemented fix: Keep the user reason where the replay can read it, or send no note on replay.

N-059 | RESOLVED P2
The new generic 4xx branch in src/http.ts puts the library's own message into the client answer

Finding: Chat added a branch: any thrown object with a numeric status 400-499 (other than 401/403/404/429) returns `type: validation_failed`, `title: err.message`. A 413 or 405 is typed validation_failed, and message text of third-party errors can reach clients. The owner's own branch maps by class only. No test covers it. http.ts is a shared file (D-02).

Implemented fix: Do not use `err.message`: map status to catalogue titles for HttpException and Fastify errors only, with a test per status; better, send the change to the owner and revert chat's edit.

Batch 13: N-098 - N-102

Five-issue review group

N-098 | PENDING VALIDATION P2
One muter still slows the room about 10x (cause not found)

Finding: With one muter of 200 authors, frames are batched and no send fails, but p95 is 1.7 to 2.2 s against 159 to 175 ms in control runs.

Reported evidence: 3 of 3 muted runs were slow; 4 of 4 controls at 100 msg/s were fast. The host was busy, so this is a pattern, not a proof.

Required action: Re-run muted and control steps in turn on a quiet host after the fix above.

Audit status: Pending - required benchmark evidence is missing.

N-099 | RESOLVED P2
Member search and next pages freeze the server

Finding: Only the default first page of the member list is cached. Every other page sorts the whole room per call, and all messages wait meanwhile.

Implemented fix: Keep one sorted list per room, or cache the sorted list for 1 to 2 seconds, and slice it.

N-100 | RESOLVED P2
In overload, sends get no clear answer and some are stored without an ack

Finding: Past the limit, sends end as a generic internal_error or get no answer in 60 s. There is no typed busy answer.

Implemented fix: Refuse early with a typed retryable server_busy, always send clientMessageId, and keep transaction timeouts under the ack timeout.

N-101 | RESOLVED P2
A database blip at connect looks like being signed out

Finding: Any handshake error that is not a refusal is sent to the browser as unauthenticated, so a short database error reads as a lost session.

Implemented fix: Map these errors to a retryable reason and keep unauthenticated for real sign-in failures.

N-102 | PENDING VALIDATION P2
Speed at 2,500 to 5,000 online swings between pass and fail

Finding: The same step can pass in one run and miss the target by seconds in the next. The cause is not proven; the host was shared and busy.

Reported evidence: 5,000 online x 100 msg/s passed 1 of 3 runs. 3,000 x 100 failed 3 of 3. 2,000 to 2,500 x 50 failed 2 of 2.

Required action: Repeat on a quiet Linux host with GC and pool metrics.

Audit status: Pending - required benchmark evidence is missing.

Batch 14: N-060 - N-064

Five-issue review group

N-060 | RESOLVED P3
The idempotency fingerprint omits paid ad fields

Finding: is18Plus and currencies are not in the fingerprint. The same key with a different choice replays the first ad instead of answering 409.

Implemented fix: Add both fields to the fingerprint.

N-061 | RESOLVED P3
Ad maintenance refund loads every ad and ignores the budget

Finding: The maintenance refund has no page and no time limit. It loads all ads in one go, so a big backlog can run past its budget.

Implemented fix: Page the query and stop at the time budget.

N-062 | RESOLVED P3
Pinned minutes have edge-case errors

Finding: Pause uses ceil, so a 10-second show costs a full minute of queue time. After RESET_RESUME the old DONE slots still count as used, so a later refund treats the fresh run as spent.

Implemented fix: Count exact seconds on pause, and exclude slots from before the reset.

N-063 | RESOLVED P3
Hidden fallback in rewardFor

Finding: When everyUsd is 0 under DYNAMIC, the code pays the full amount for any spend. Nobody granted this fallback.

Implemented fix: Refuse when everyUsd is 0.

N-064 | RESOLVED P3
Unanswered gifts can block the ones behind them

Finding: A full page of UNKNOWN answers releases the claims and ends the pass, and the next pass takes the same rows. Later gifts are never asked.

Implemented fix: Order by last-asked time.

Batch 15: N-065 - N-069

Five-issue review group

N-065 | RESOLVED P3
Revert scans have no time index

Finding: Each page filters COMPLETED rows by time with only a status index, so each page sorts all matching rows.

Implemented fix: Add a partial index on (created_at, id) WHERE status = COMPLETED.

N-066 | RESOLVED P3
Expired gifts stay PENDING_VERIFICATION for ever

Finding: expired is only worked out on read and no sweep changes the row. The admin list shows them as waiting.

Implemented fix: Add a sweep to CANCELLED, or show expired in the admin list.

N-067 | RESOLVED P3
The gift KYC gate uses the chat entitlement and a guessed level

Finding: Sending gifts is decided by publicChat, and minLevel is the user's level plus one. Scope lists Transfer KYC limits as a Transfer feature, so this may not belong in chat.

Implemented fix: Remove the guessed level and leave the check to the Transfer provider, if the owner agrees.

N-068 | RESOLVED P3
Gift confirm does not re-check a ban or gift restriction placed after start

Finding: start checks ban and RESTRICT_GIFTS, and confirm up to 10 minutes later does not. Scope lets pending transactions finish before maintenance or off, but that line does not cover bans.

Implemented fix: Repeat the ban and restriction check in confirm, if the owner agrees.

N-069 | RESOLVED P3
Rain cleanup and small defects

Finding: Dead code remains (buildLegs and its tests, unused db methods and services), with a hard-coded BTC, ETH, USDT list and a float feePercent. After a long OFF the old pot and price pay at once, and a retried MANUAL_SYSTEM job ends FAILED though paid. System interval allows SECONDS, which scope does not list, and the scheduler hides a missing-table error by message text.

Implemented fix: Delete the dead code and list, keep fee maths exact, drop a stale pot, and treat the unique-key hit as already paid.

Batch 16: N-070 - N-074

Five-issue review group

N-070 | RESOLVED P3
A bad DM cursor silently restarts at page one

Finding: A cursor that does not parse is ignored and the list restarts, so a client can loop on the first page with no error.

Implemented fix: Refuse a bad cursor with a 400.

N-071 | RESOLVED P3
Admin rain code has copies and an unbounded list

Finding: formatMinor, record, view and small helpers are copied, and the reason minimum differs (1 and 3). One route returns every recipient with no paging, though a Rainpool may have 100,000. changeRestriction logs no old value, unlike changeBan.

Implemented fix: Share the helpers, page the recipients, and log the old value.

N-072 | RESOLVED P3
New gift columns have no check rule or rollback note

Finding: kind and revert_shortfall have no CHECK, unlike every other status column, and the folder has no rollback note.

Implemented fix: Add the check rules and the rollback note.

N-073 | RESOLVED P3
The same reason code cannot be in both report lists

Finding: report_type was added but the key stays on code alone, so SPAM cannot exist for both MESSAGE and USER.

Implemented fix: Key on (code, report_type), or accept the limit.

N-074 | RESOLVED P3
Who added a room entry is lost on admin writes

Finding: Admin inserts leave out created_by_admin, and room_excluded_users has no such column.

Implemented fix: Write the admin id, and add the column if the action log does not cover it.

Batch 17: N-075 - N-079

Five-issue review group

N-075 | RESOLVED P3
Soft-delete ids break the house rule

Finding: New soft-delete tables default to gen_random_uuid, and schema.prisma says never to use it. Harm is low because the tables are small.

Implemented fix: Use the app id, in line with the house rule.

N-076 | RESOLVED P3
Retry ids are deleted

Finding: Maintenance prunes client_message_ids and dm_message_dedupe, while V2-67 says keep every chat table for ever with listed exceptions.

Implemented fix: Keep them, or list them as an exception.

N-077 | RESOLVED P3
The hiders index keeps an empty Set per author forever

Finding: unload deletes the user from the author's Set but never deletes the Set when it is empty, so memory grows with every author ever hidden by an online user.

Implemented fix: Delete the Set when empty.

N-078 | RESOLVED P3
Facts load is not de-duplicated

Finding: Two parallel calls for one user (join and HTTP list) both run the full facts load; the staff path adds dbNow and a restriction lookup.

Implemented fix: Keep an in-flight Promise map per user.

N-079 | RESOLVED P3
Option routes swallow every error and skip the live-user check

Finding: giftOptions wraps the user lookup in try/catch and returns an anonymous 200 with fewer options on a DB or Redis failure, although the route is behind AuthGuard (the comment is wrong). adOptions skips getChatUser, so a disabled account with a live session can read it.

Implemented fix: Remove the catch; call getChatUser in adOptions.

Batch 18: N-080 - N-084

Five-issue review group

N-080 | RESOLVED P3
Ad 'kind' values disagree between the migration and the code

Finding: The migration comment and default say USER/INTERNAL, the code writes COMMERCIAL and only tests !== 'INTERNAL'. There is no CHECK constraint, so three names mean the same thing.

Implemented fix: Pick one set of values and add a CHECK constraint.

N-081 | RESOLVED P3
Restriction cache edge cases after invalidation

Finding: After a 'restrictions' event sets the cache to null, a failed reload returns no rows, so all restrictions lift until the next read (the comment promises the opposite). A reload in flight during the event stores old rows for up to 30 s.

Implemented fix: Keep the last rows on invalidate and add a generation check.

N-082 | RESOLVED P3
User stats count refunded and cancelled ads as spent

Finding: The spentUsd shown to staff sums all ads without filtering status or netting refunds.

Implemented fix: Filter by status or net out refunds.

N-083 | RESOLVED P3
A report stays open after its request is approved

Finding: The staff request carries no report id, so the report stays PENDING; a second moderator gets already_banned. In that REPORT flow the report also flips to ACTIONED for a moment.

Implemented fix: Carry the report id in the request and close the report on approval.

N-084 | RESOLVED P3
Limitations page leaves out group and room-list restrictions

Finding: The query uses target_type = 'USER' only, so a user under a group ban or a room-list entry sees nothing while being refused. A room-scoped RESTRICT_DM is accepted silently but never applies.

Implemented fix: Include group and room-list rows; refuse or apply room-scoped RESTRICT_DM.

Batch 19: N-085 - N-089

Five-issue review group

N-085 | RESOLVED P3
Production code that exists only for a test mock

Finding: A typeof check falls back to getRooms when the test double lacks getActiveRoomIds; getRooms no longer counts messages, so that branch is also wrong for large lists.

Implemented fix: Delete the branch and give the test double the method.

N-086 | RESOLVED P3
Fake fields in the moderation-settings reply and report stats

Finding: Ids are array positions (they shift on removal), createdAt is an empty string, and thankYouSent is always null.

Implemented fix: Store an id and time per restriction (or return no such fields) and drop thankYouSent until data exists.

N-087 | RESOLVED P3
History of deleted accounts disappears from admin lists

Finding: The ban list joins identity.users with deleted_at IS NULL and the staff list does too; history messages show 'Unknown'. Chat keeps every row and should show 'deleted user'.

Implemented fix: Left join and show 'deleted user'.

N-088 | RESOLVED P3
DM silent failures: swallowed errors and a pin limit that cannot be reached

Finding: Follow handler, time-zone port errors and merchant port errors are swallowed with no log (and the app id is shown as its name); the pin limit is counted from a list capped at 100 so a max above 100 never triggers. The forward loop (stops on a non-Problem error) and the stranger/app cap race are already FR-204 and FR-171.

Implemented fix: Log and count the swallowed errors; use `count(*)` for the pin limit; see FR-204 and FR-171 for the rest.

N-089 | RESOLVED P3
Hidden defaults in DM code: last-seen NOBODY, typing switch ON, 60 s notice window, search length twice

Finding: Several values act without an admin setting: `?? 'NOBODY'` for last seen, the typing switch column defaults to ON, a 60 s notice window, search length 2..100 written twice, and comments say id and login code are accepted for lookup. Two logo mappers exist (FR-127).

Implemented fix: Read them from admin settings.

Batch 20: N-090 - N-094

Five-issue review group

N-090 | RESOLVED P3
Invite leftovers: accepted invite stays pending, closed groups count toward the cap, expired shown as revoked

Finding: Joining a group where the user is already a member ignores `already_member`, so the invite stays PENDING and counts toward the invitee's cap; the pending cap counts invites to closed groups; expired invites are stored as REVOKED.

Implemented fix: Return a distinct result and close the invite; join `closed_at IS NULL` in the cap count; use an EXPIRED state.

N-091 | RESOLVED P3
Two admins editing group permissions lose each other's change

Finding: The service reads the permission object, builds the full object and writes it whole with no lock.

Implemented fix: Do it in a transaction with `lockGroup`, or merge the JSON in SQL.

N-092 | RESOLVED P3
Docs hold fixed sample codes (123456)

Finding: The admin Postman says "the development code is 123456", the gift confirm example uses `{"code":"123456"}` (V2-85: payment PIN only) and the rain confirm example uses the same. No real secret was found. A risk only if a production build accepts these values.

Implemented fix: Replace with a placeholder and remove the sentence; update with the PIN change.

N-093 | RESOLVED P3
Hard-coded coin list in rain balances

Finding: When the user's currency list is "all", rain balances falls back to `['BTC','ETH','USDT']` typed in code. Not in the granted fallbacks.

Implemented fix: Read the coin list from the wallet/currency port or settings; refuse when unavailable.

N-094 | RESOLVED P3
Dead exports, unused injections, misleading comments and a stale database default

Finding: Unused injected `relations`, `delivery`, `bansDb`, `ports` in the group service; `UPLOADS_DIR`/`UPLOADS_URL_PREFIX` exports with a false "main.ts imports them" comment; a module comment saying TwoFactorModule answers gift codes; `RestrictionsCoreModule`, `SessionDbOperation`, `RainEmailCode`, `isVpnOrProxy` provided for rain tiers nothing calls; comments with empty references "(Module 9,)" and removed claims; dead `newDirectCount` and `StoredPrefs.timeZone`; a revert comment promising "revert will not be conducted" when funds are short (opposite of V2-83); misleading reason `not_a_member` for "not banned".

Implemented fix: Delete the unused code and fix the comments listed.

Batch 21: N-095 - N-096

Final partial group: 2 findings

N-095 | RESOLVED P3
openapi.json and the chat Postman point at port 3005; the platform default is 3001

Finding: The server URL was changed from 3001 to 3005 in chat's commit while config.ts and .env.example use 3001, so "try it" and Postman hit a port nothing listens on. Carried since v1 and never listed as a shared-file edit.

Implemented fix: Restore 3001 or set the chat Postman variable only.

N-096 | RESOLVED P3
Admin room list returns a constant 0 as the message count

Finding: To remove the slow count (FR-012) the SQL now selects `0::bigint AS "messageCount"` and `toAdminRoom` passes it on, so the admin sees "0 messages" in every room.

Implemented fix: Drop the field or fill it from the partition estimate.

Completion gate

The code changes for 100 findings are committed and pushed. This audit remains incomplete while N-098 and N-102 lack quiet Linux performance evidence. Capture repeated muted/control and 2,500-5,000 online runs with GC and pool metrics, then update these statuses and reissue the audit.

Source: OREBIT-CHAT-RAIN-FIXES.html. Repository revision fe95d59 on branch index. Audit snapshot date: 2026-11-02.